

BHTTP/1: a binary framing for HTTP

Specification, version 1

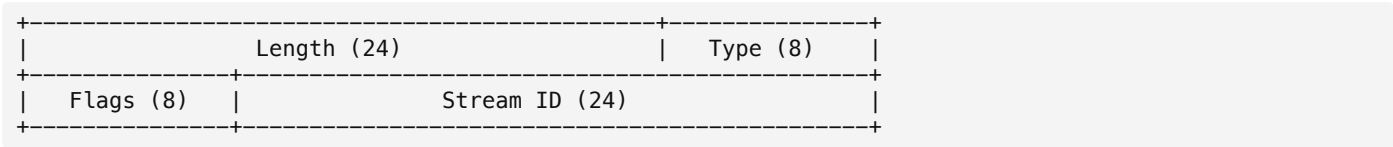
MUST, MUST NOT, SHOULD, RECOMMENDED and MAY are used as in RFC 2119 and RFC 8174. Integers are unsigned and big-endian. Sizes are in bytes.

1. Connection

BHTTP/1 runs over one TCP connection (default port 9000). The server answers requests in order on the connection they came in on. A client sends all of one invocation's requests on one connection and MUST NOT open a second. If it closes early, the client reports an error rather than reconnecting. A client SHOULD wait for each response before sending the next request, but a server MUST also accept pipelined requests. It reads each request completely before answering it. A server MAY refuse a connection by closing it at once; otherwise it closes only after a connection error (§6), after a body it cannot finish (§4), when a request takes too long to arrive or the client stops taking response frames, or when no request has been in progress for a while (at least 10 s RECOMMENDED; unknown frames do not count as activity). Unless a body fails or the client stops taking frames, it never closes with a fully received request unanswered, so a request with no response was not processed.

2. Frame header

Every frame is an 8-byte header followed by Length bytes of payload.



- Length: payload size, not counting the header (0 to 16 777 215).
- Type: 0x00 DATA, 0x01 HEADERS. Any other value is an unknown type (§3).
- Flags: 0x01 END_STREAM marks the last frame of a request or response. Senders MUST set the other bits to 0 and receivers MUST ignore them.
- Stream ID: the client numbers its requests on a connection, starting at 1. Each request MUST use a higher ID than the one before (gaps are allowed), and the response carries the same ID. Stream 0 stands for the connection itself: in v1 it carries only unknown frames and the connection-error message of §6. A client never uses an ID above 16 777 215; one with more requests stops and reports an error.

3. Unknown frame types

A receiver that reads a frame of unknown type MUST skip it: read and discard exactly Length payload bytes, then go on to the next frame. It MUST NOT reply to it, close the connection because of it, or let it change any request state, whatever its stream ID or flags (END_STREAM on an unknown frame ends nothing). This holds everywhere in the frame sequence, including between the frames of one request or response, and it overrides every error rule below. Types 0xF0–0xFF will never be assigned, so senders can use them to test that peers skip; 0x02–0xEF are left for later versions. HTTP/2 has the same rule outside its header blocks (RFC 9113 §4.1, §5.5).

4. Messages

A request is one HEADERS frame and then zero or more DATA frames on the same stream, with END_STREAM on the last frame. A response has the same shape. A message with no body is a single HEADERS frame with END_STREAM set. The header block MUST fit in one HEADERS frame of at most 65 535 bytes, and a receiver MUST accept any such block, however many fields it holds. Senders SHOULD keep DATA frames to 16 384 bytes or less; receivers MUST accept any Length. A body ends at END_STREAM. A response to HEAD is always a single HEADERS frame with END_STREAM. Any other response with content-length MUST carry exactly that many DATA bytes; a difference is a protocol error. A request's content-length MUST be well-formed (§5) and is otherwise ignored. A sender that cannot finish a body MUST close the connection without END_STREAM, so the client can tell the body is incomplete.

5. Header block

A HEADERS payload is a sequence of fields with no count and no terminator; the frame's Length bounds it. In the grammar, tag is one byte and len is one or two bytes.

field	=	tag [name] value	; name present only when tag = 0
name	=	len bytes	value = len bytes
len	=	0xxxxxxx	; 0 to 127
		1xxxxxxx xxxxxxxx	; 0 to 32 767 (15 bits, big-endian)

Tag 0 means the name follows as a literal. Tags 1 to 10 name the fields of a plain GET exchange, listed below. Tags 11 to 255 are not defined in v1.

tag	name	tag	name	tag	name	tag	name	tag	name
1	:method	3	:status	5	user-agent	7	server	9	content-type
2	:path	4	host	6	accept	8	date	10	content-length

- Literal names are 1 to 32 767 lowercase HTTP token characters (RFC 9110 §5.6.2), so pseudo-headers (names starting with :) can only be sent by tag. Senders MUST use the tag for a table name; receivers MUST accept non-pseudo table names as literals too.
- Values are 0 to 32 767 bytes and may hold any byte except NUL, CR and LF. content-length is 1 to 18 ASCII digits. date SHOULD use the HTTP date format (RFC 9110 §5.6.7) and is not checked. Senders SHOULD use the shortest len; receivers accept both.
- Pseudo-headers come before all other fields. :method, :path, :status, host and content-length appear at most once each, counted by name whether sent by tag or literal; other names may repeat.
- A request has one :method, a case-sensitive HTTP token, and one :path, starting with / and at most 1024 bytes as sent. It has no :status; host SHOULD be sent.
- A response has one :status, three ASCII digits from 200 to 599, and no :method or :path. v1 has no 1xx responses.
- A block breaking any of these rules, using tags 11 to 255, or with a length running past the payload is malformed.

6. Server

The server cuts :path at the first ? or # and decodes %XX escapes (%2F is a slash, %2e%2e is ..). A directory, or a path ending in /, means its index.html. Every response carries content-type, content-length, server and date; a found file gets 200 and the file as DATA, an error a short text/plain body, and HEAD no DATA. The first matching row below decides. A segment is a piece of the decoded path between slashes; resolved means after following symbolic links.

#	status	when
1	400	malformed header block (§5), block over 65 535 bytes, bad % escape (not two hex digits) or %00
2	405	a well-formed method other than GET or HEAD
3	403	a segment equal to ..
4	404	any other segment starting with a dot (., .env)
5	403	the resolved path lies outside the root, or the server may not read it
6	404	the resolved path has a component starting with a dot, or is not a regular file
7	500	the file cannot be opened or read for any other reason

These are stream errors: the response goes on the request's stream (after any request body is read and discarded) and the connection stays open. Connection errors, where the frame sequence can no longer be trusted, are: HEADERS on stream 0 or with an ID not higher than the last; DATA with no request open or on a different stream; or a new HEADERS before the open request's END_STREAM. After answering requests completed before the bad frame, the server sends the connection-error message on stream 0 (HEADERS with :status 400, then DATA holding the reason, the last with END_STREAM) and closes. If a frame is cut off by EOF, the server just closes.

7. Client

The client matches responses to requests in order. A HEADERS frame on stream 0, at any point, starts a connection-error message; the client reads it and its DATA, reports the reason and stops. These are protocol errors, after which the client closes the connection: a HEADERS or DATA frame for any stream other than the oldest unanswered request; DATA before HEADERS or after END_STREAM; a second HEADERS in one response; DATA in a response to HEAD; a malformed or oversized response header block; a content-length mismatch; a frame cut off by EOF; and a read timeout.

8. Design notes

HTTP/2's header is Length 24, Type 8, Flags 8, then a reserved bit and a 31-bit Stream ID: 9 bytes. Its drafts used 8 bytes (a 16-bit Length in draft 04, 14 bits plus two reserved bits in draft 13) until draft 14 (2014) widened Length to 24 bits so a receiver can opt in to larger frames for bulk transfer; the default stays 2^{14} so one big frame cannot delay the others. Flags are defined per frame type, so 8 bits are enough. The 31-bit Stream ID is a 32-bit word minus a reserved bit with no meaning (RFC 9113 §4.1); both ends start streams (client odd, server even), giving the client 2^{30} IDs and the server $2^{30} - 1$.

BHTTP/1 uses 24 / 8 / 8 / 24 (8 bytes):

- Length 24. v1's sizes (16 KiB DATA recommended, header blocks at most 64 KiB – 1) fit in 16 bits; the extra byte allows larger frames now and in v2. Its width can never change, since §3's skipping relies on every version reading Length alike.
- Type 8 and Flags 8. Whole bytes need no bit masking, and with Length they fill the first 32-bit word exactly, the Stream ID the second. v1 uses two types and one flag. A v2 can add frame types freely (§3), but a v1 peer would ignore a new flag and reject a new tag, so those need agreement first (below).
- Stream ID 24. v1 answers strictly in order and has no server-started streams, so it needs no odd/even split or reserved bit. The ID still matches each response to its request, exposes stale or misplaced frames, and leaves room for v2 multiplexing. At the 19 000 requests a second bserve manages on loopback, 2^{24} IDs last 15 minutes of continuous use.

From HPACK (static and dynamic tables, Huffman coding, prefix-coded integers) BHTTP/1 takes two ideas: names from a fixed table (HPACK's "literal with indexed name") and length-prefixed literals. It indexes names only, since most values here (paths, dates, lengths) change on every message.

With no preface, a misdirected HTTP/1.1 GET / reads as an unknown frame of Length 0x474554 and is skipped until the idle limit; v1 accepts that rather than add a preface to every connection. To grow, a v2 client first sends a new frame type, which a v2 server acknowledges before its first response and a v1 server skips.